



# 算法设计与分析

讲授内容：贪心法

2025年11月3日

- 最小生成树
- 最优子结构
- 贪婪选择
- Prim's 贪婪 MST算法

# 最小生成树

**输入:** 一个连通的, 无向图  $G = (V, E)$   
其加权函数  $w: E \rightarrow \mathbb{R}$   
为了简化, 假设所有边的权各不相同.

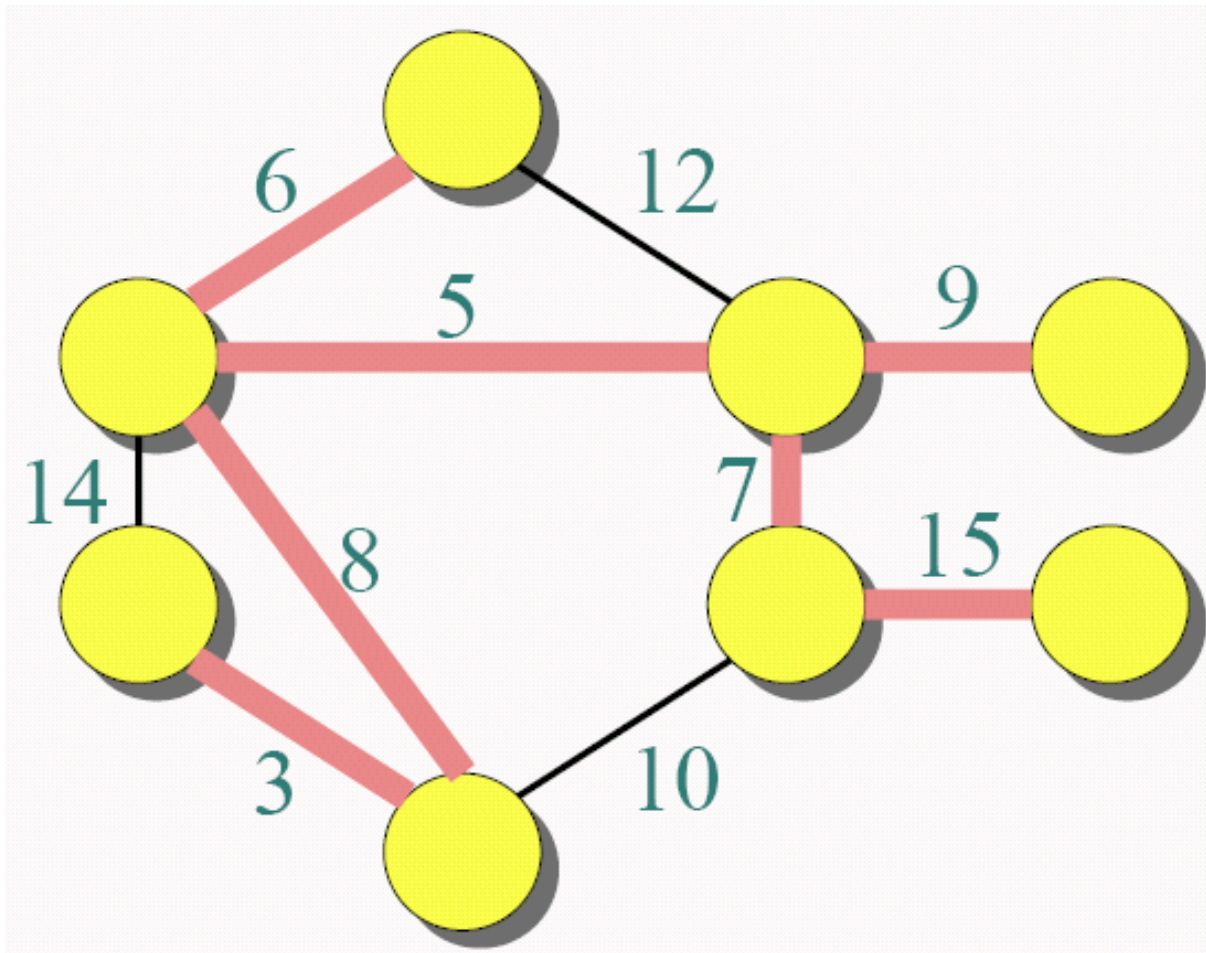
**输出:** **生成树**  $T$ —连接所有顶点的树  
— 其权最小:

$$w(T) = \sum_{(u,v) \in T} w(u,v).$$

问题:

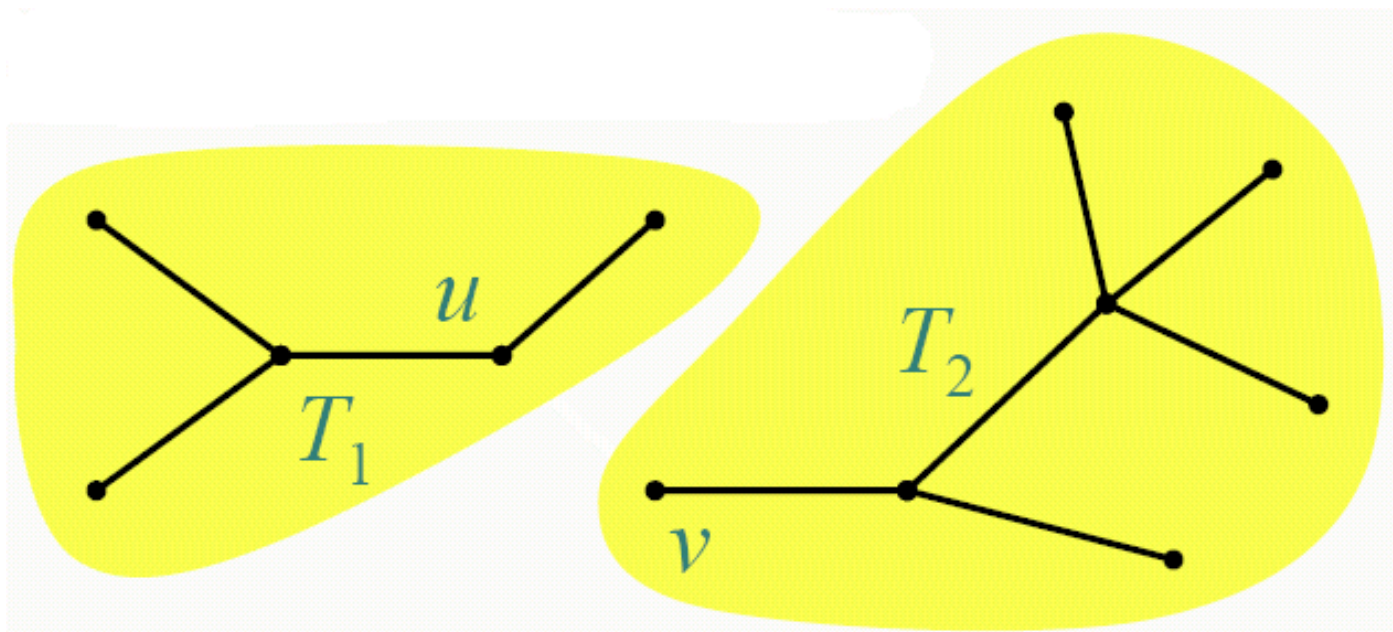
- 是否可以使用动态规划算法求解?
- 是否可以使用贪心算法求解?
- 算法效率的区别?

# MST (Minimum Spanning Tree) 举例



# 最优子结构

MST  $T$ :  
( $G$ 的其他顶点没有画出)



去掉边  $(u, v)$   $T$ . 然后, 将  $T$  划分为两棵子树  $T_1$  和  $T_2$ .

**定理:** 子树  $T_1$  是  $G_1 = (V_1, E_1)$  的MST, 其中,  $G_1$  是由  $T_1$  的顶点**导出**的  $G$  的子图。

$$V_1 = T_1 \text{ 的顶点,}$$
$$E_1 = \{ (x, y) \in E : x, y \in V_1 \}.$$

$T_2$  类似.

# 证明最优子结构

**证明：** 粘贴拷贝： $w(T) = w(u, v) + w(T_1) + w(T_2)$ .  
如果  $T_1$  是  $G_1$  中比  $T_1$  加权更小的扩展树，那么在  $G$  中  $T = \{(u, v)\} \cup T_1 \cup T_2$  将是一棵比  $T$  加权更小的扩展树。

我们得到了重叠子问题了吗？

- 将图中的顶点排序  $\{v_1, v_2, \dots, v_i, \dots, v_n\}$ 。  $w(i)$  表示连接  $\{v_1, v_2, \dots, v_i\}$  的最小生成树的权值。  
很好，那么可以使用动态规划！
- 是的，但是 MST 表现出更强特征，可以使用更加有效的算法。

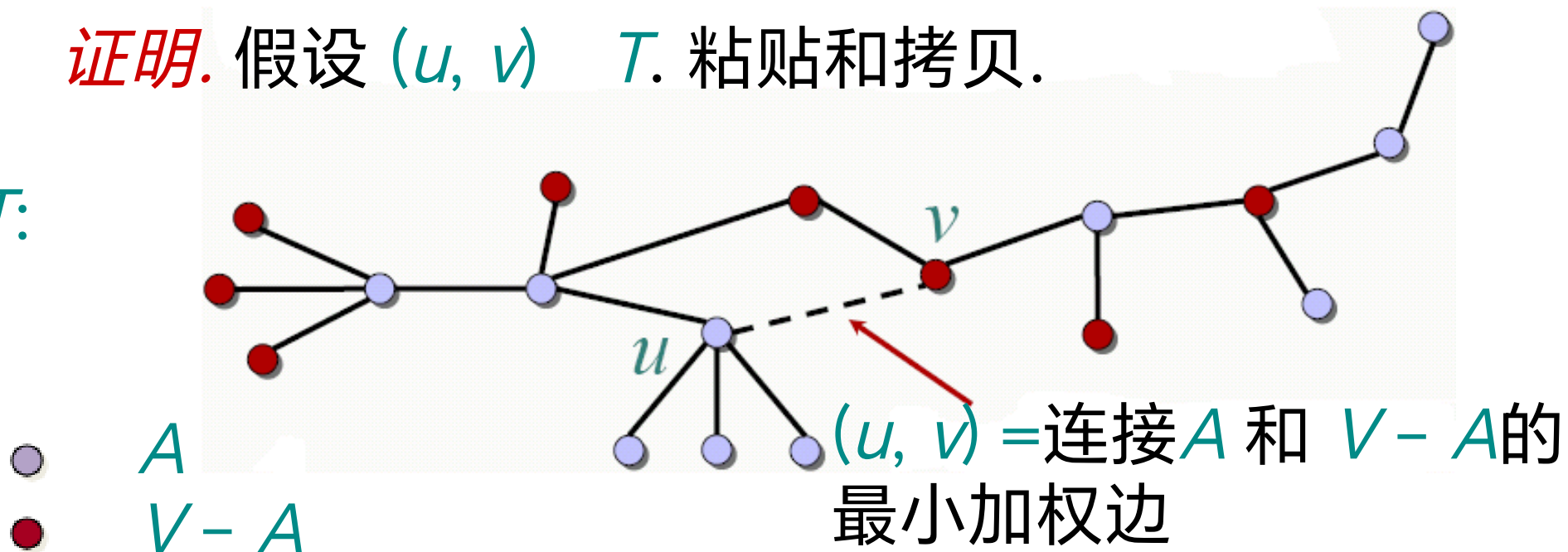
**贪婪选择特征**  
局部的最优选择  
全局范围内也是最优的.

**定理：**令  $T$  为  $G = (V, E)$  的 MST, 并且令  $A \subseteq V$ .  
假设  $(u, v) \in E$  是连接  $A$  和  $V - A$  的最小加权边.  
那么,  $(u, v) \in T$ .

# 定理的证明

**证明.** 假设  $(u, v)$   $T$ . 粘贴和拷贝.

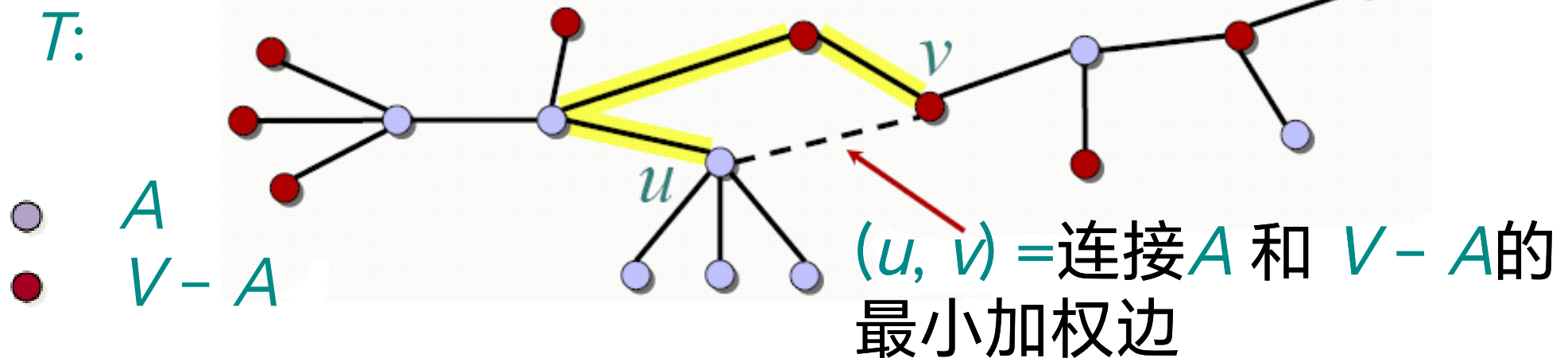
$T$ :





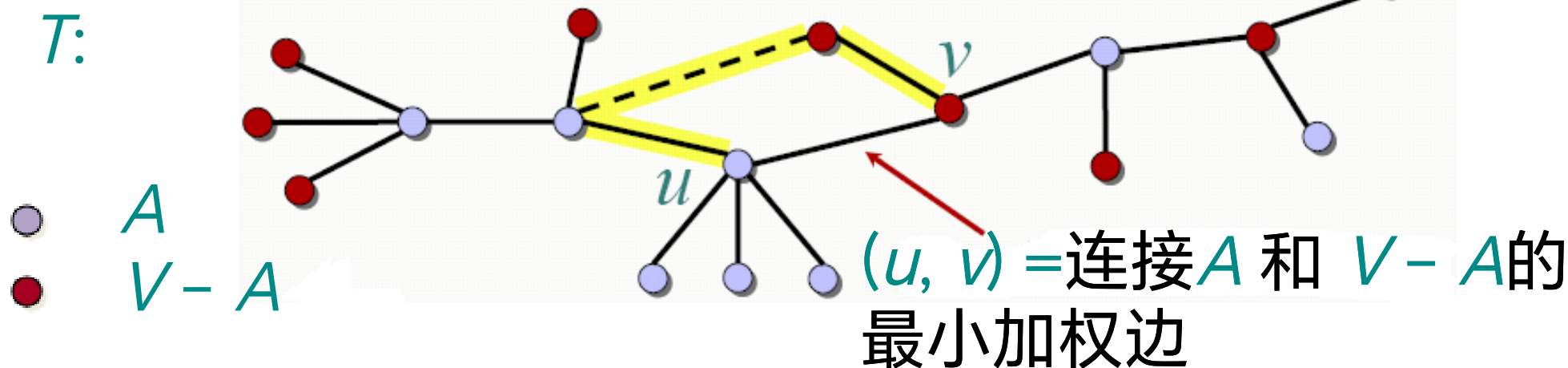
# 定理的证明

**证明.** 假设  $(u, v)$   $T$ . 粘贴和拷贝.



考虑  $T$  中从  $u$  到  $v$  的唯一的简单路径.

**证明.** 假设  $(u, v)$   $T$ . 粘贴和拷贝.

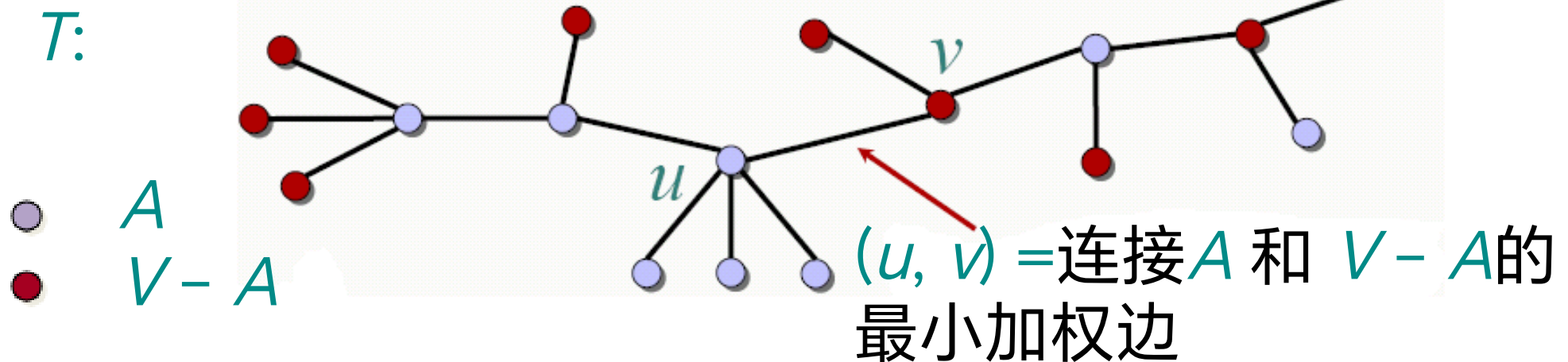


考虑  $T$  中从  $u$  到  $v$  的唯一的简单路径.

将  $(u, v)$  和这条路径上的第一条边交换, 这个边连接  $A$  中的一个顶点, 同时连接  $V-A$  中的一个顶点。

# 定理的证明

**证明.** 假设  $(u, v) \in T$ . 粘贴和拷贝.



考虑  $T$  中从  $u$  到  $v$  的唯一的简单路径.

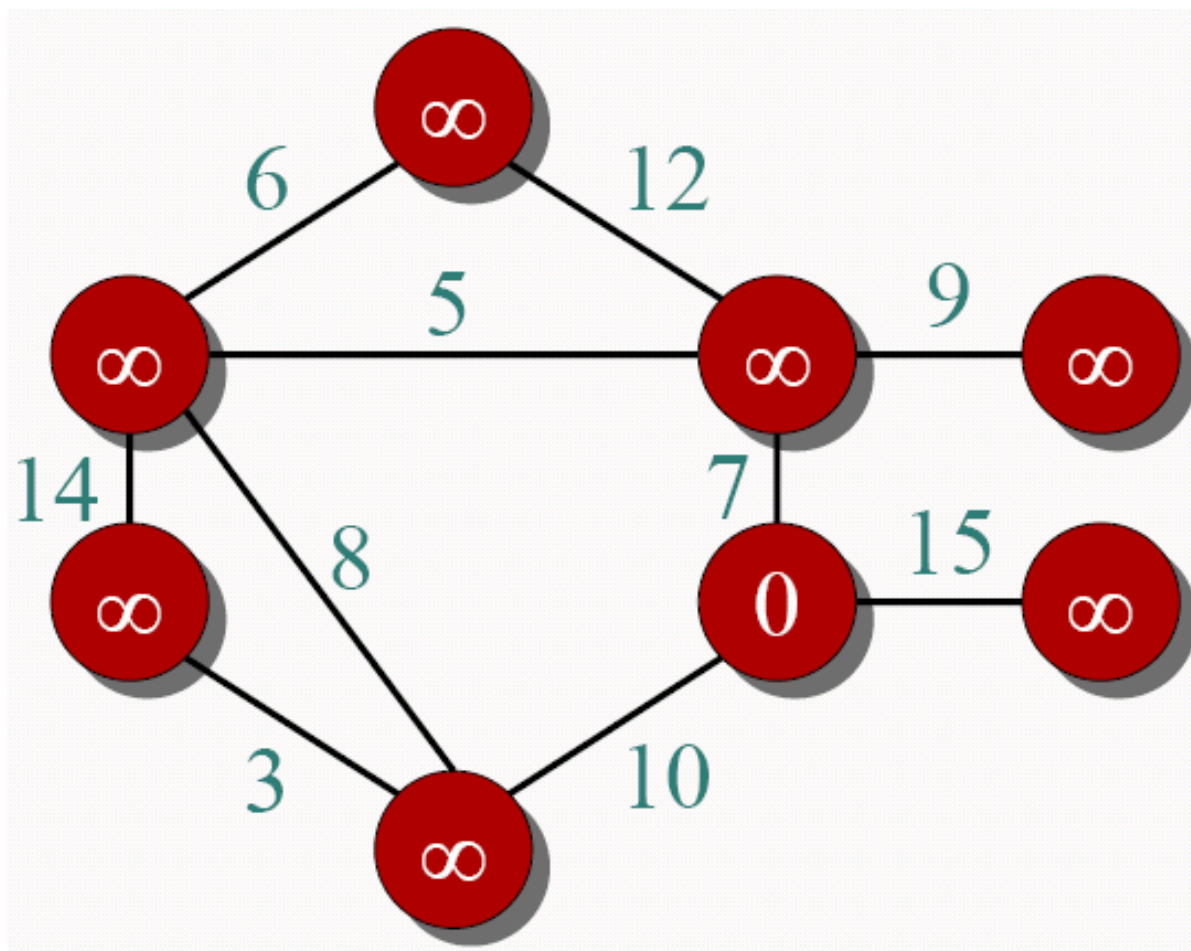
将  $(u, v)$  和这条路径上的第一条边交换, 这个边连接  $A$  中的一个顶点, 同时连接  $V - A$  中的一个顶点. 一个比  $T$  加权更小的生成树产生了.

**定理:** 令  $T$  为  $G = (V, E)$  的 MST, 并且令  $A \subseteq V$ . 假设  $(u, v) \in E$  是连接  $A$  和  $V - A$  的最小加权边. 那么,  $(u, v) \in T$ .

**思路:** 用优先队列  $Q$  维护  $V - A$ . 将  $Q$  中的每个顶点按照其和  $A$  中的顶点连接的边的最小权进行排序。

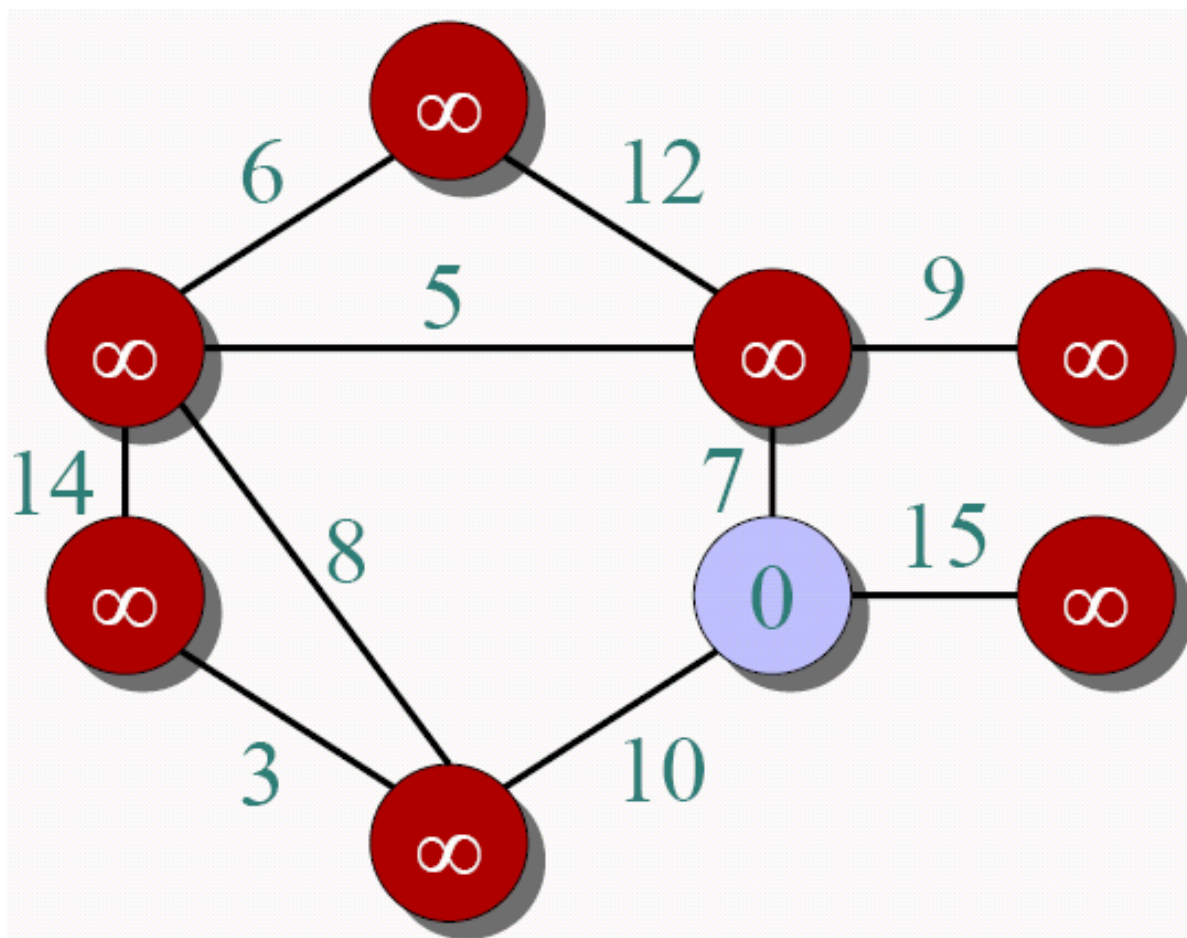
# Prim算法举例

●  $\in A$   
●  $\in V - A$



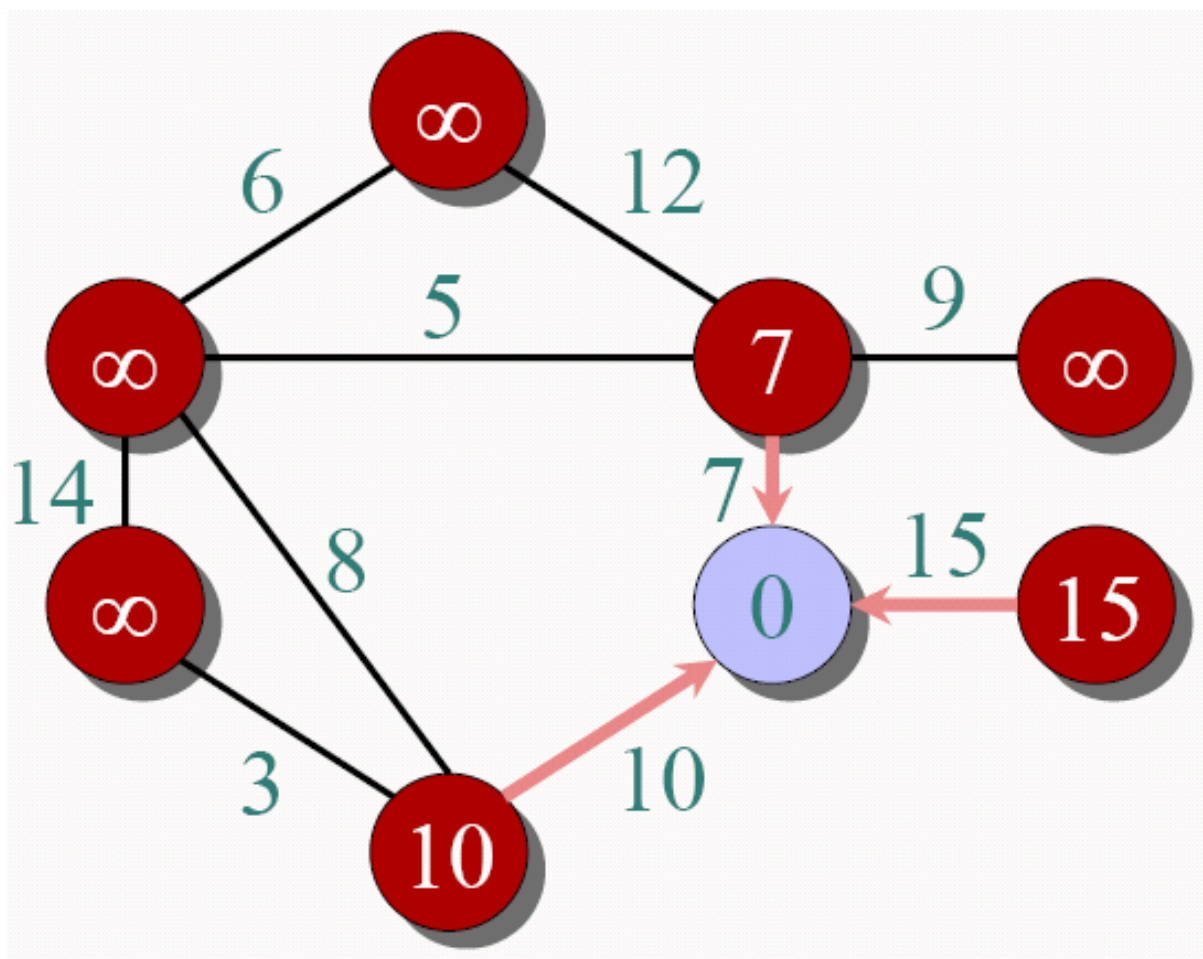
# Prim算法举例

●  $\in A$   
●  $\in V - A$



# Prim算法举例

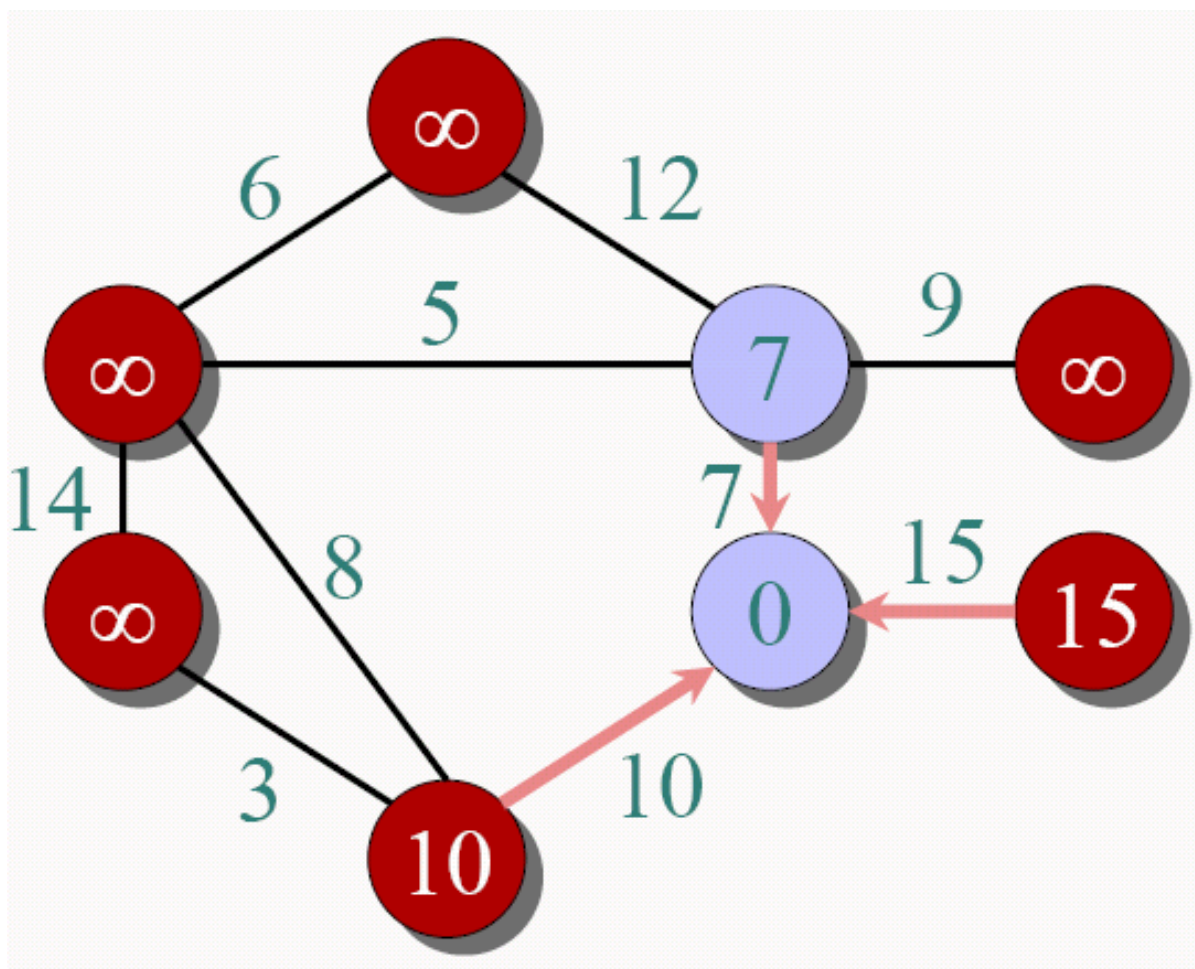
●  $\in A$   
●  $\in V - A$





# Prim算法举例

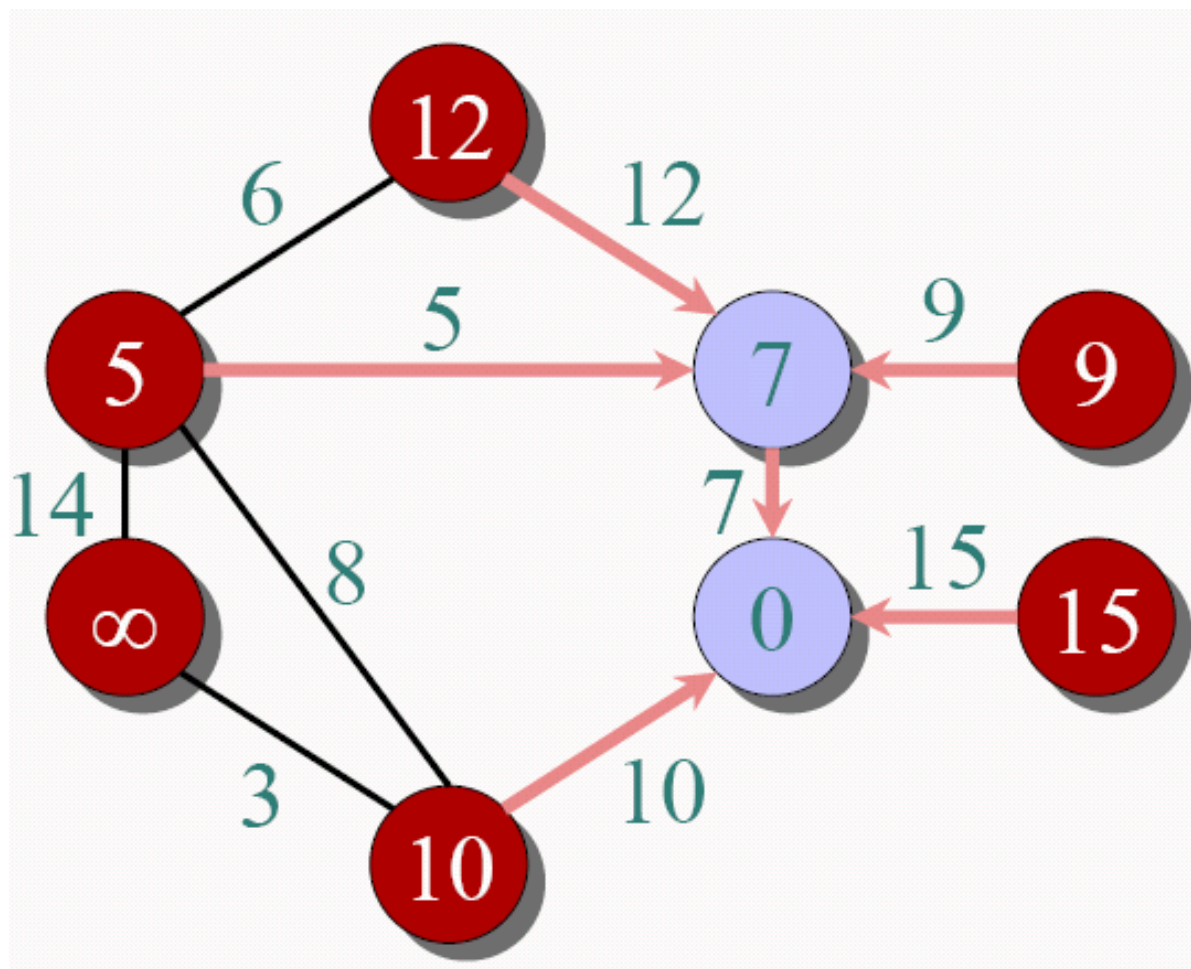
●  $\in A$   
●  $\in V - A$





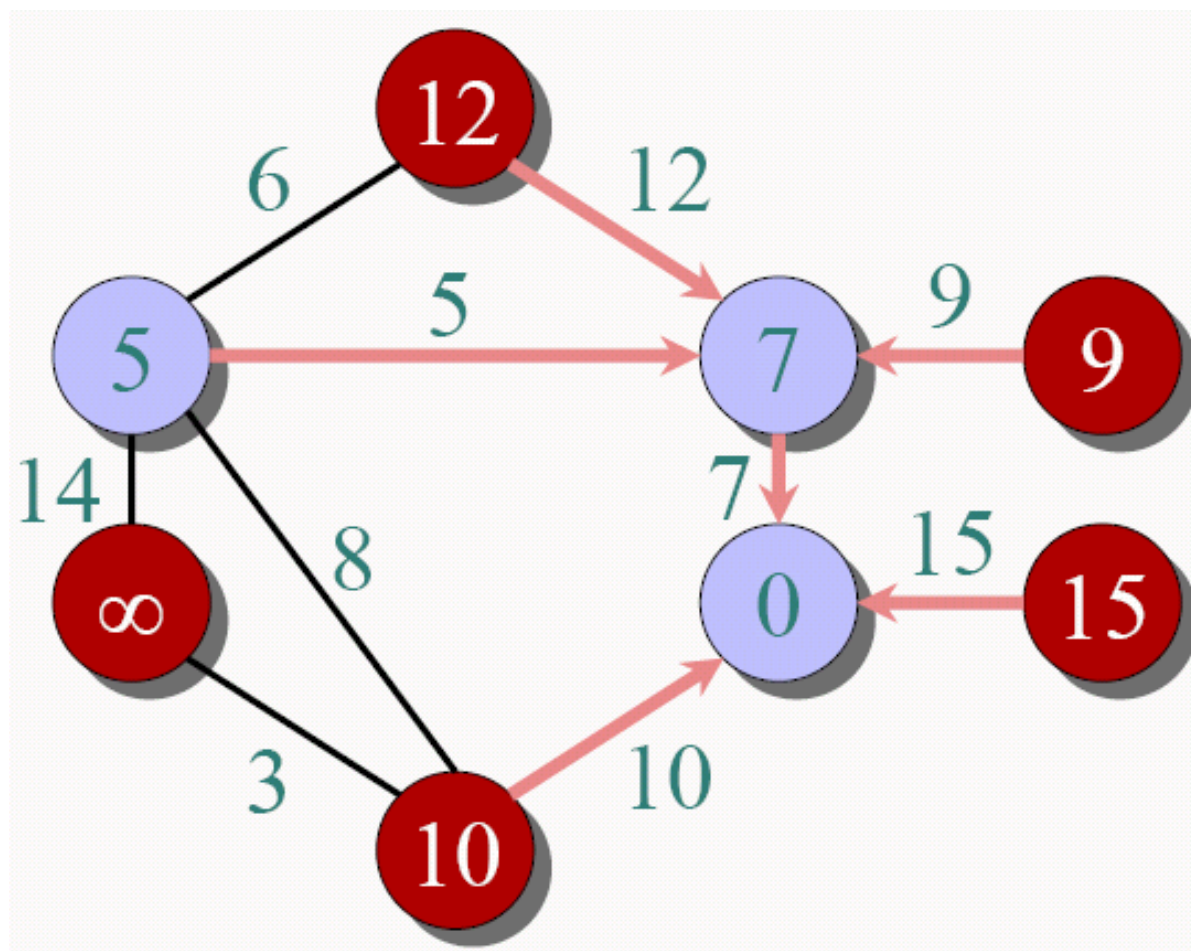
# Prim算法举例

●  $\in A$   
●  $\in V - A$



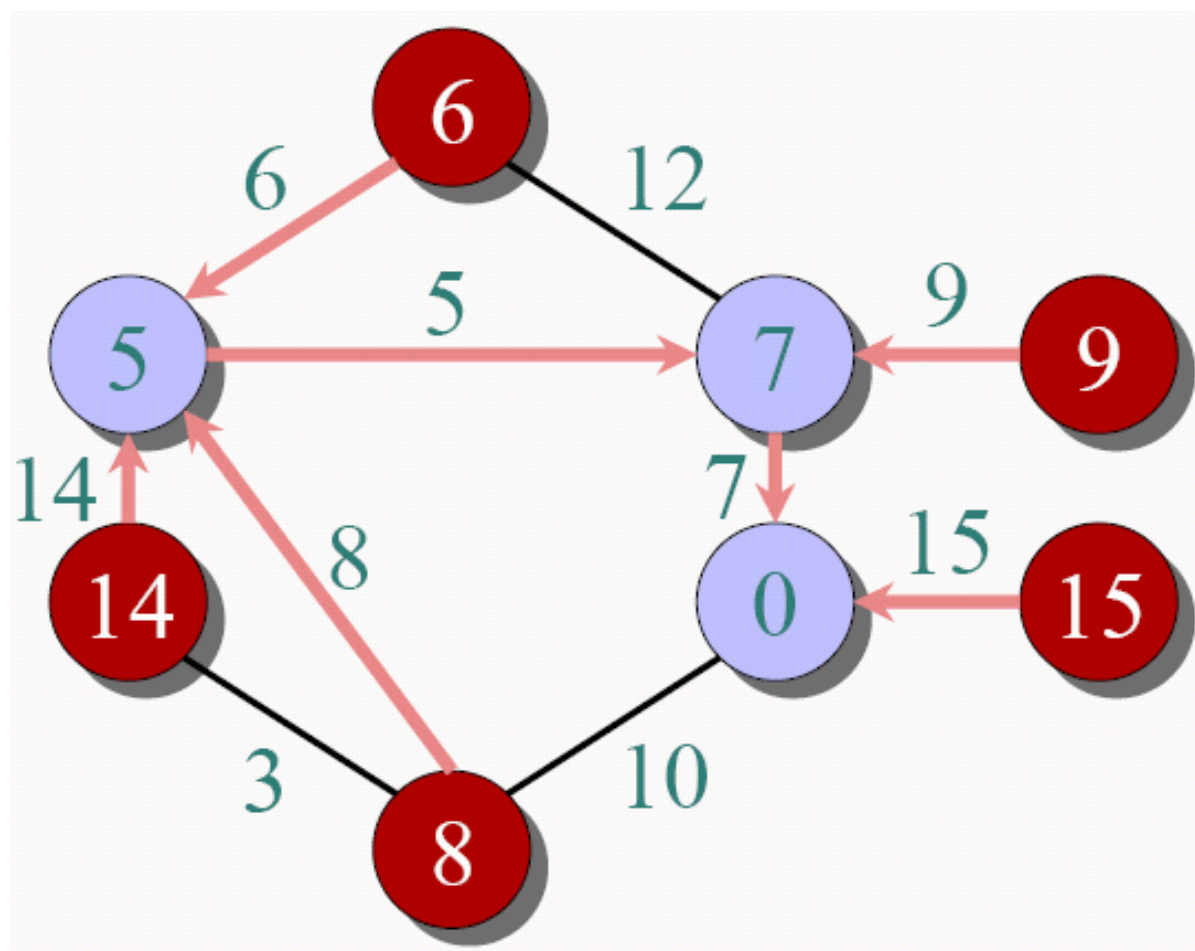
# Prim算法举例

●  $\in A$   
●  $\in V - A$



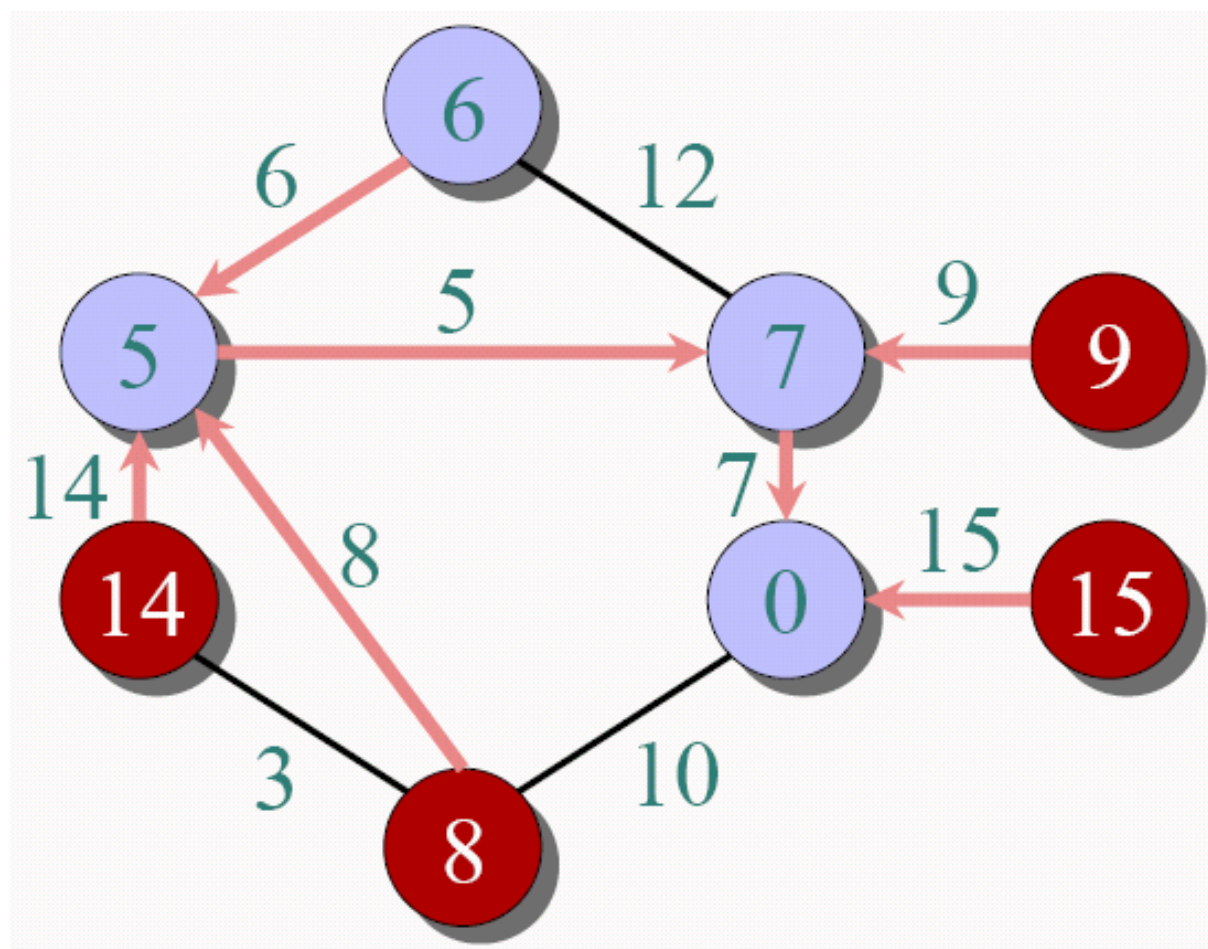
# Prim算法举例

●  $\in A$   
●  $\in V - A$



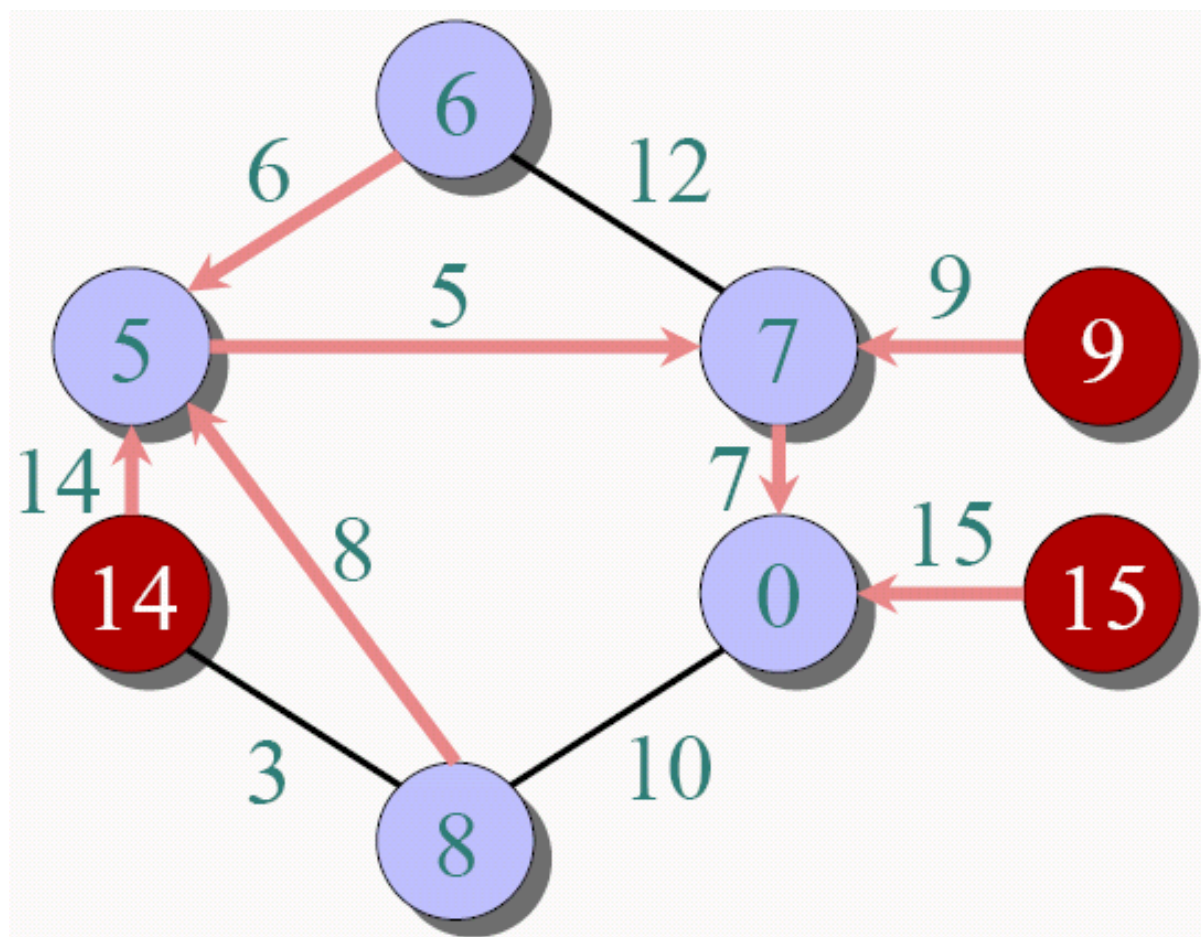
# Prim算法举例

●  $\in A$   
●  $\in V - A$



# Prim算法举例

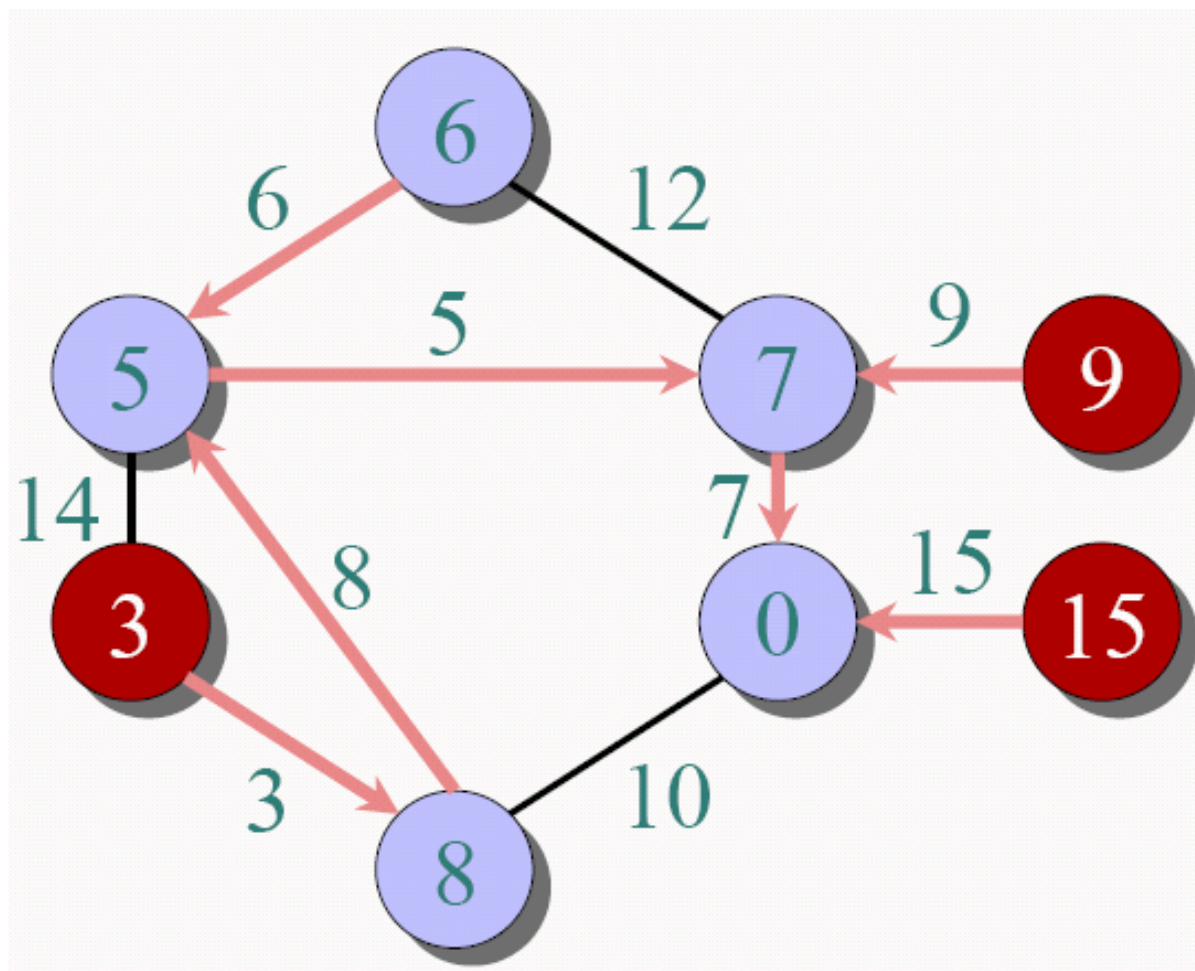
●  $\in A$   
●  $\in V - A$





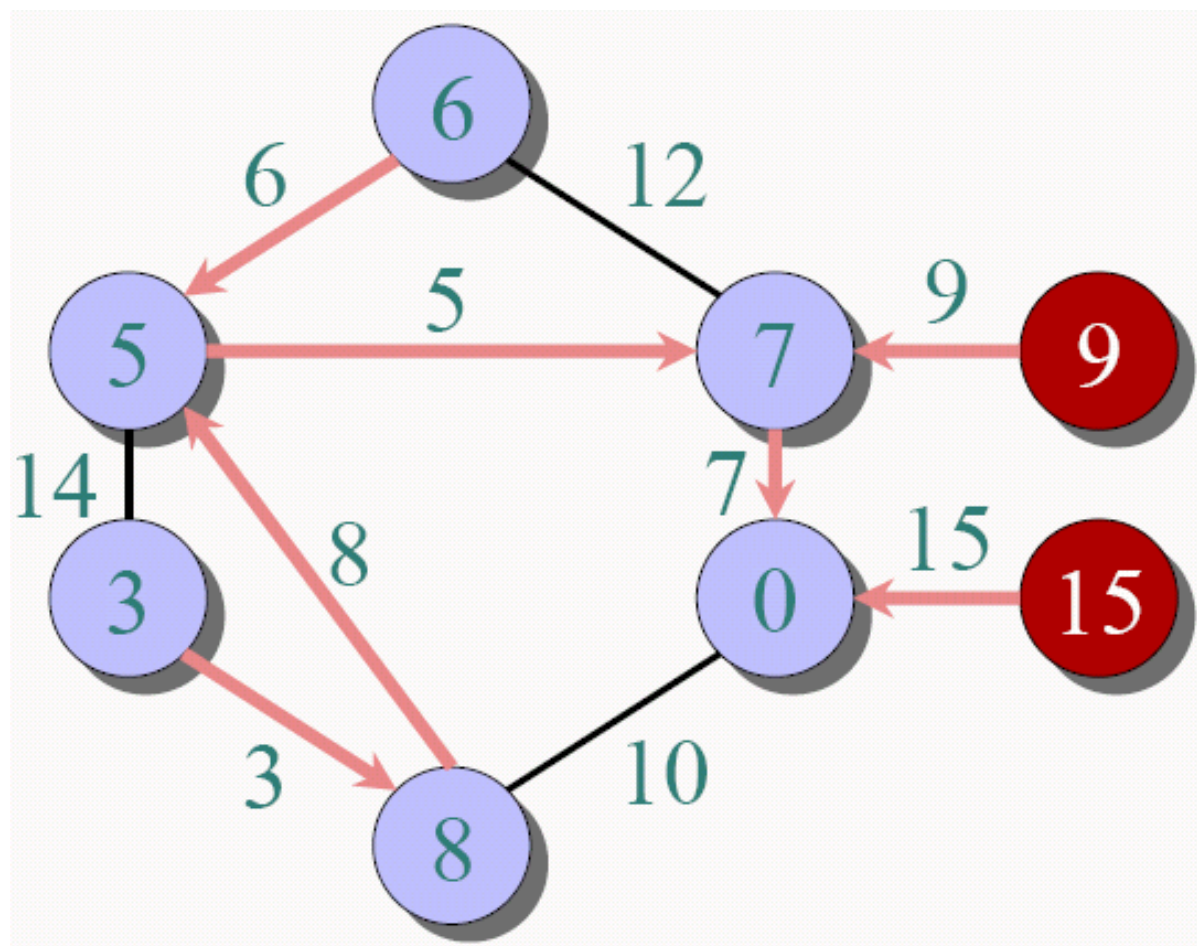
# Prim算法举例

●  $\in A$   
●  $\in V - A$



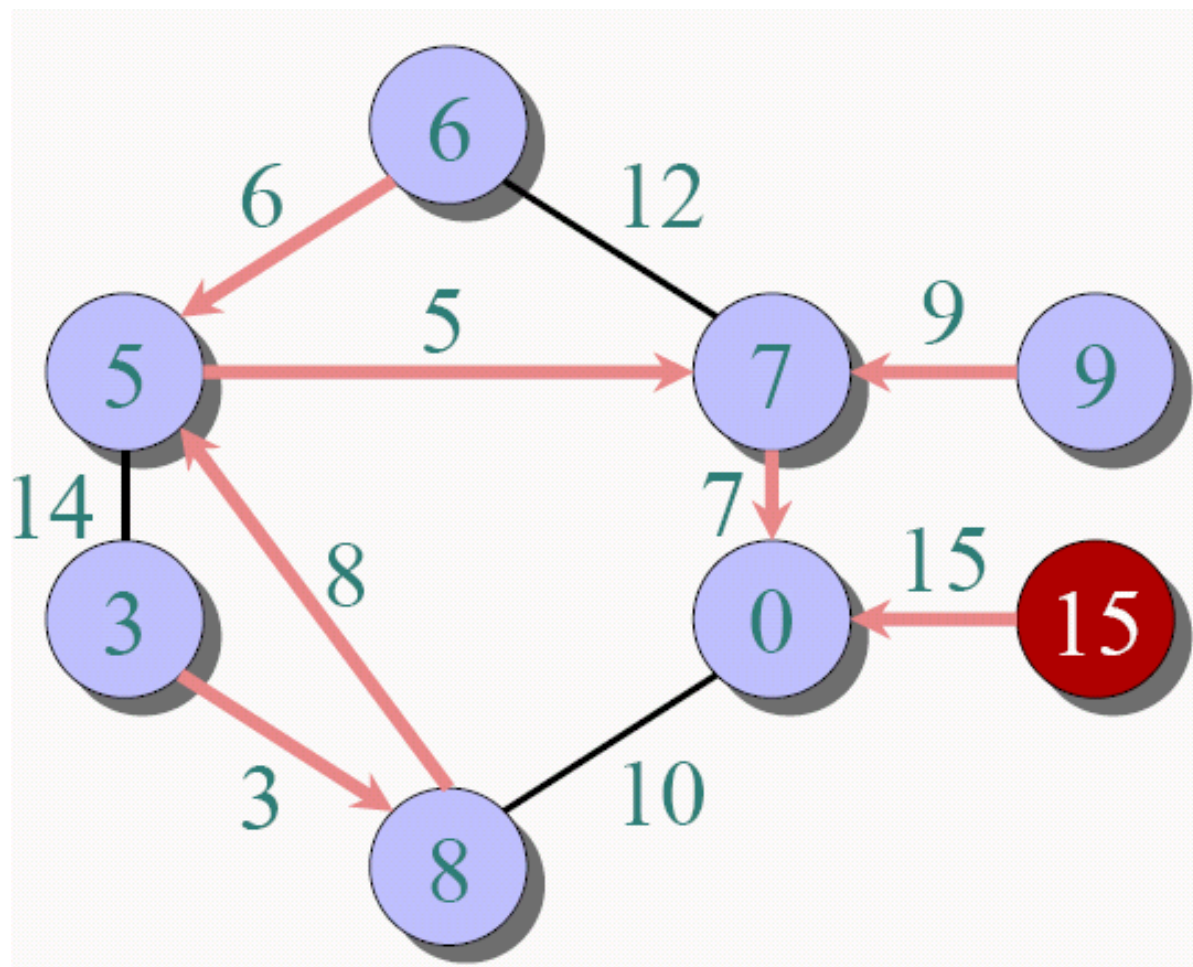
# Prim算法举例

●  $\in A$   
●  $\in V - A$



# Prim算法举例

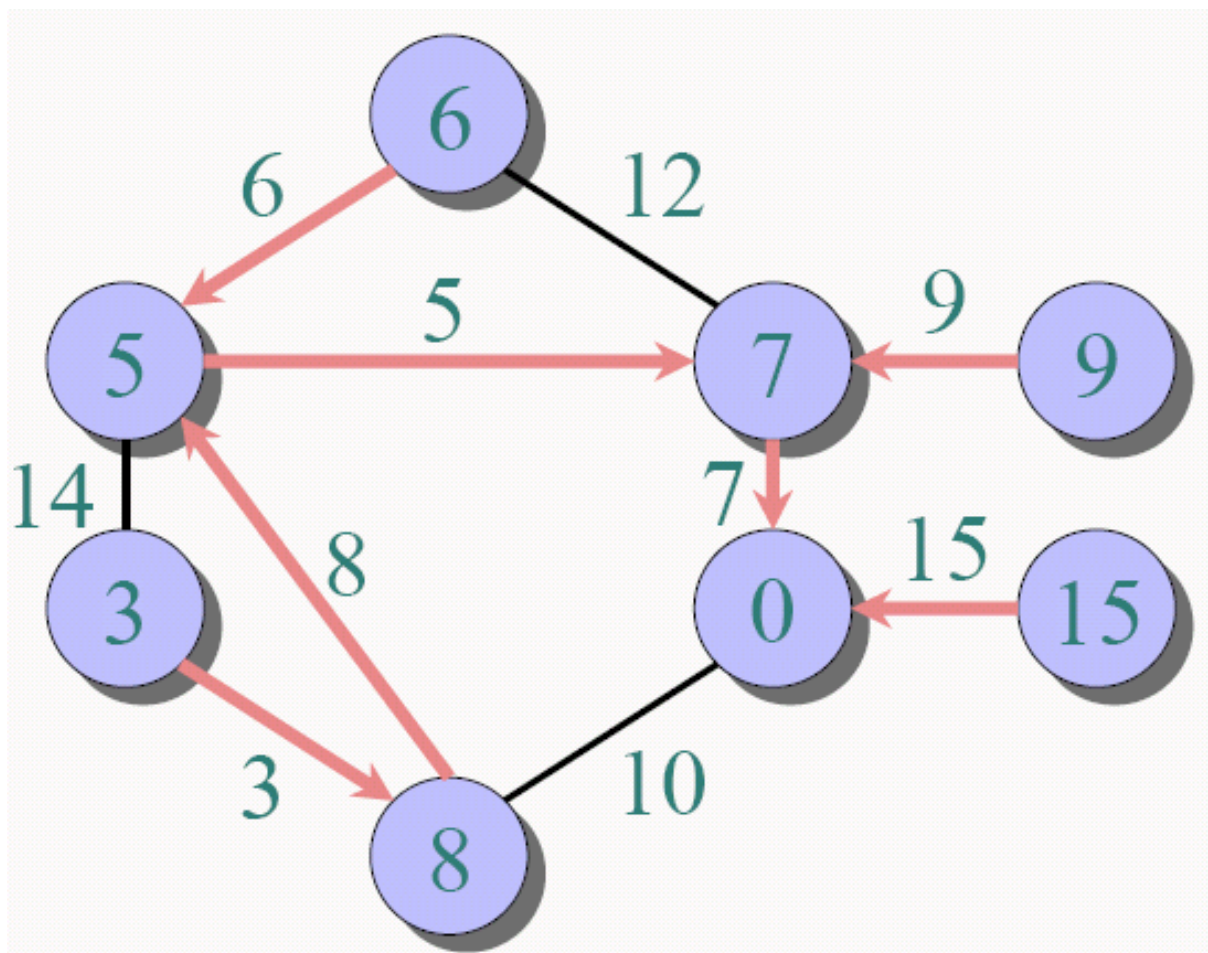
●  $\in A$   
●  $\in V - A$





# Prim算法举例

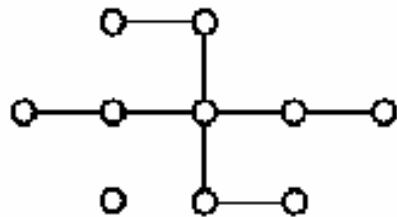
●  $\in A$   
●  $\in V - A$



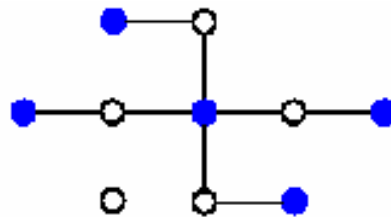
- 什么时候使用贪婪算法？
  - **最优子结构**: 问题的最优解包含了其子问题的最优解。
    - 例如, 如果  $A$  是  $S$  的最优解, 那么  $A' = A - \{1\}$  是  $S = \{i \in S: s_i \leq f_1\}$  的最优解.
  - **贪心选择特性**: 全局的最优解可以通过局部的最优 (贪婪) 选择得到。
    - 动态规划需要检查子问题的解。
- 贪心算法 (试探) 并不能总是得到最优解.
- 谈论算法和动态规划 (DP) 对比
  - 相同: 最优子结构
  - 差别: 贪婪选择特性
  - 如果贪婪算法不是最优的, 可以使用 DP。

- 活动选择问题
- 背包问题
- 哈夫曼编码

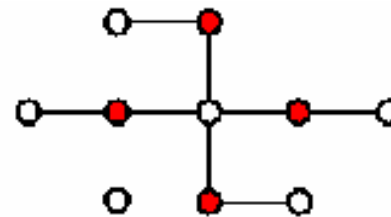
- 无向图  $G=(V, E)$  的一个**顶点覆盖**为一个子集  $V' \subseteq V$ , 使得如果  $(u, v) \in E$ , 那么  $u \in V'$  **或者**  $v \in V'$ , 或者两者都是.
  - 顶点覆盖的集合包括所有的边.
- 一个顶点覆盖的大小就是这个覆盖着顶点的数目。
- 顶点覆盖问题就是找到一个图的最小的顶点覆盖。
- 贪婪试探: 每次覆盖尽量多的边 (有最大度的边) 然后删除所有被覆盖的边。
- **贪婪试探并不能总是找到最优解!**
  - 顶点覆盖问题是 NP完全的.



一个图



贪婪算法得到的大小为5的顶点覆盖

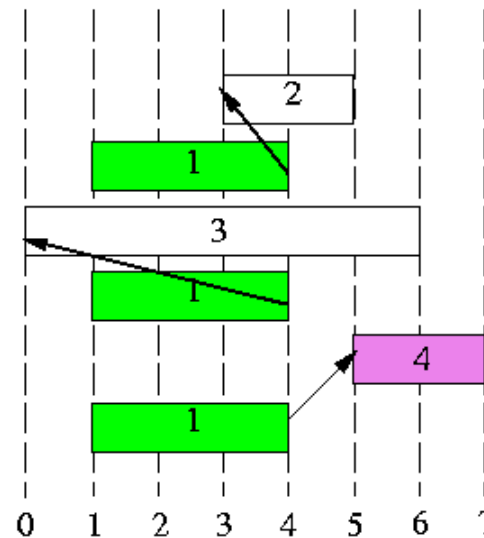


一个大小为4的最优顶点覆盖

- 什么时候使用贪婪算法？
  - **贪心选择特性:** 全局的最优解可以通过局部的最优（贪婪）选择得到。
    - 动态规划需要检查子问题的解。
  - **最优子结构:** 问题的最优解包含了其子问题的最优解。
    - 例如, 如果  $A$  是  $S$  的最优解, 那么  $A' = A - \{1\}$  是  $S' = \{i \in S: s_i \neq f_1\}$  的最优解。
- 贪心算法 (试探) 并不能总是得到最优解。
- 谈论算法和动态规划 (DP) 对比
  - 相同: 最优子结构
  - 差别: 贪婪选择特性
  - 如果贪婪算法不是最优的, 可以使用DP。

- **活动选择问题:** 给定一个集合  $S = \{1, 2, \dots, n\}$   $n$  个计划的活动, 对每个活动  $i$ , 开始时间为  $s_i$  结束时间为  $f_i$ , 选择出相互兼容的活动最大集合.
  - 如果被选中, 活动  $i$  在半开放的区间  $[s_i, f_i)$  中进行.
  - 活动  $i$  和  $j$  **兼容** 如果  $[s_i, f_i)$  和  $[s_j, f_j)$  不重叠 (i.e.,  $s_i \geq f_j$  or  $s_j \geq f_i$ ).

| $i$ | $s_i$ | $f_i$ |
|-----|-------|-------|
| 1   | 1     | 4     |
| 2   | 3     | 5     |
| 3   | 0     | 6     |
| 4   | 5     | 7     |



Compatible activities:  
(1, 4), (2, 4)

- 子问题空间

$S_{ij} = \{a_k \in S: f_i \leq s_k < f_k \leq s_j\}$  (第*i*第*j*个活动之间的活动)

虚构活动 $a_0$ 与 $a_{n+1}$ , 令 $f_0=0$ 、 $f_{n+1}=\infty$ , 于是 $S=S_{0,n+1}$

- 假设 $f_0 \leq f_1 \leq f_2 \leq \dots \leq f_n < f_{n+1}$

当 $i \geq j$ 时,  $S_{ij} = \emptyset$

- 最优子结构

- 令 $S_{ij}$ 的最优解为 $A_{ij}$

- $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$



- $$C[i, j] = \begin{cases} 0, & \text{如果 } S_{ij} = \emptyset \\ \max_{i < k < j, a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{如果 } S_{ij} \neq \emptyset \end{cases}$$
- 由此可以设计一个动态规划求解方法
- 问题：活动选择问题是否可以使用贪心算法求解？

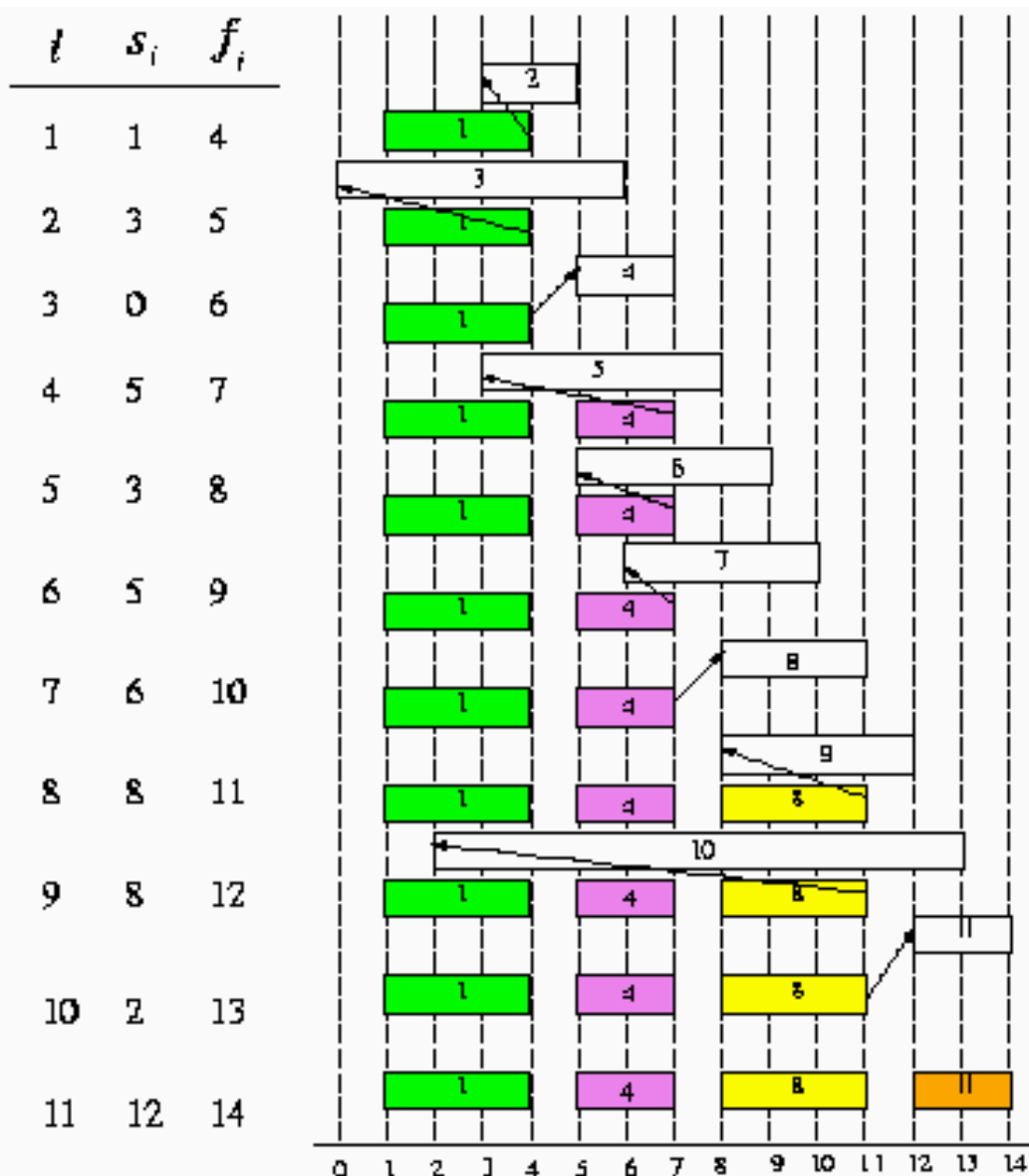
- 基本思想：选出活动后，剩下的资源应能被尽量多的其他任务所用【选择最早结束的活动】
- 定理：对于任意非空子问题 $S_{ij}$ ，设 $a_m$ 是 $S_{ij}$ 中具有最早结束时间的活动：

$f_m = \min \{f_k : a_k \in S_{ij}\}$ ，那么

- 活动 $a_m$ 在 $S_{ij}$ 的某个最大兼容活动子集中
- 子问题 $S_{im}$ 为空，所以选择 $a_m$ 将使子问题 $S_{mj}$ 为唯一可能非空的子问题。

证毕

# 活动选择-贪心选择



1. 排序  $f_i$
2. 选择第一个活动
3. 选择第一个活动  $j$   
使得  $s_j \geq f_i$   
活动  $j$  是上一次选择的一个活动

# 活动选择问题-递归贪心算法

```
RECURSIVE-ACTIVITY-SELECTOR( $s, f, i, n$ )  
/* Assume  $f_1 \leq f_2 \leq \dots \leq f_n$ . */  
1.  $m \leftarrow i+1$ ;  
2. while  $m \leq n$  and  $s_m \leq f_i$   
3.   do  $m \leftarrow m+1$   
4. if  $m \leq n$   
5.   then return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$   
6.   else return  $\emptyset$ 
```

- 初始调用 RECURSIVE-ACTIVITY-SELECTOR( $s, f, 0, n$ )

# 活动选择问题-迭代贪心算法

```
Greedy-Activity-Selector( $s, f$ )  
/* Assume  $f_1, f_2, \dots, f_n$ . */  
1.  $n \leftarrow \text{length}[s]$ ;  
2.  $A \leftarrow \{1\}$ ;  
3.  $j \leftarrow 1$ ;  
4. for  $i \leftarrow 2$  to  $n$   
5.   if  $s_i \leq f_j$   
6.      $A \leftarrow A \cup \{i\}$ ;  
7.      $j \leftarrow i$ ;  
8. return  $A$ .
```

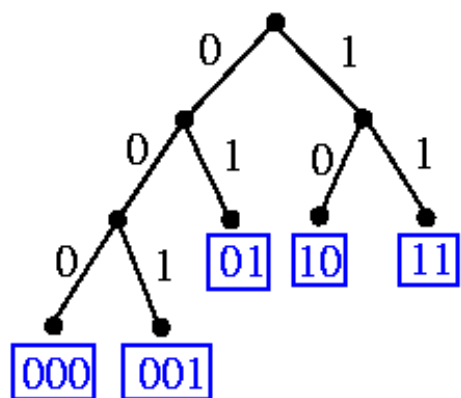
- 如果不考虑排序，算法的时间复杂度为:  $O(n)$

- 用于数据压缩, 指令集编码, 等等.
- 二进制串码: 字符用一个唯一的二进制字符串表示
  - 固定长度码 (块码): *a*: 000, *b*: 001, ..., *f*: 101    *a*ce  
000 010 100.
  - 可变长度码: 常用的的字符    短的码字; 不常用的字  
符    长的码字

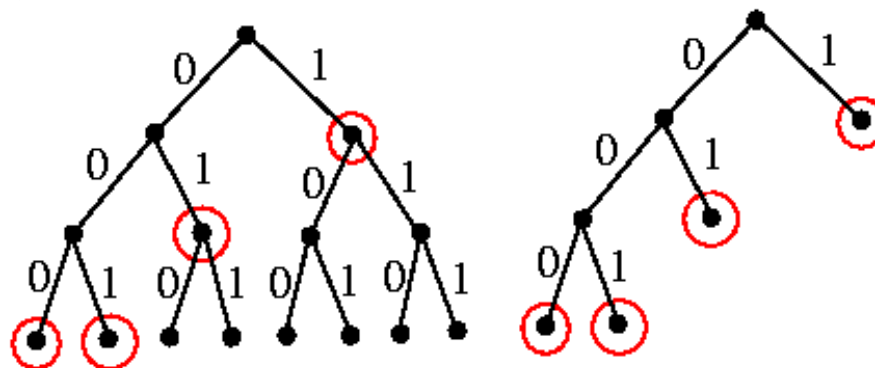
|                          | a   | b   | c   | d   | e    | f    | cost /<br>100 characters |
|--------------------------|-----|-----|-----|-----|------|------|--------------------------|
| Frequency                | 45  | 13  | 12  | 16  | 9    | 5    |                          |
| Fixed-length codeword    | 000 | 001 | 010 | 011 | 100  | 101  | 300                      |
| Variable-length codeword | 0   | 101 | 100 | 111 | 1101 | 1100 | 224                      |

# 二叉树和前缀码对比

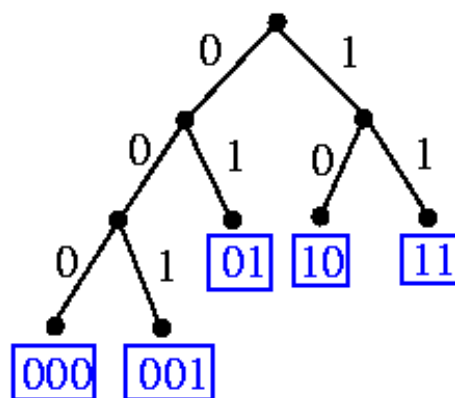
- 前缀码:任何一个字符的编码都不能是其他字符编码的前缀.



binary tree  $\rightarrow$  prefix code



prefix code {1, 01, 000, 001}  $\rightarrow$  binary tree

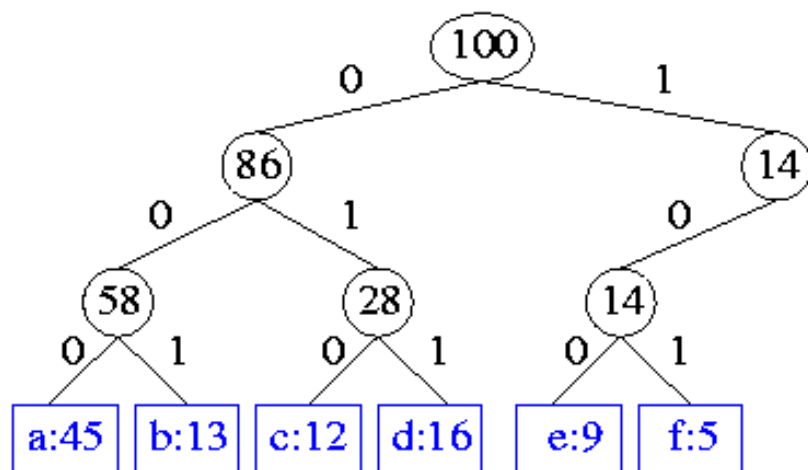


decoding: 01 10 000 000 001 11 11  
          ↑  ↑  ↑  ↑  ↑  ↑  
          arrive at a leaf, start at root



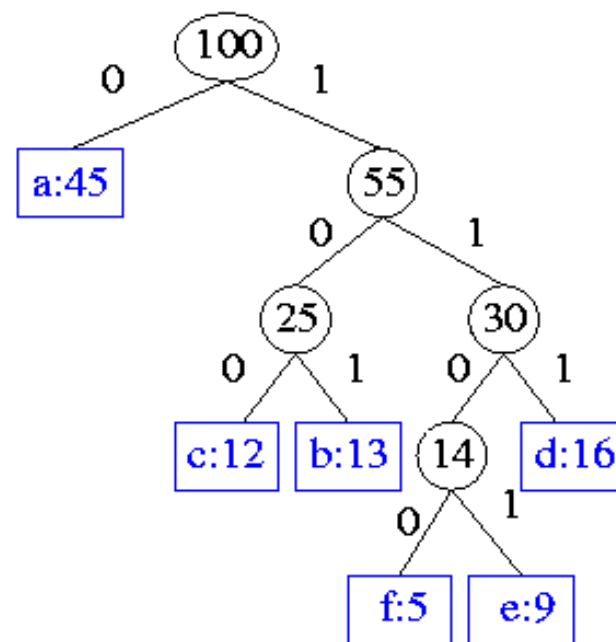
# 最优前缀码设计

- $T$ 的编码费用： $B(T) = \sum_c f(c) d_T(c)$ 
  - $c$ : 字符集  $C$  中的字符
  - $f(c)$ :  $c$  出现的频率
  - $d_T(c)$ :  $c$  的叶子的深度(码字  $c$  的长度)
- 编码设计: 给定  $f(c_1), f(c_2), \dots, f(c_n)$ , 创建二叉一棵有  $n$  个叶子的树使得  $B(T)$  最小.
  - 思路: 常用的字符使用短的深度.



Fixed-length cost:  $3 * 100 = 300$

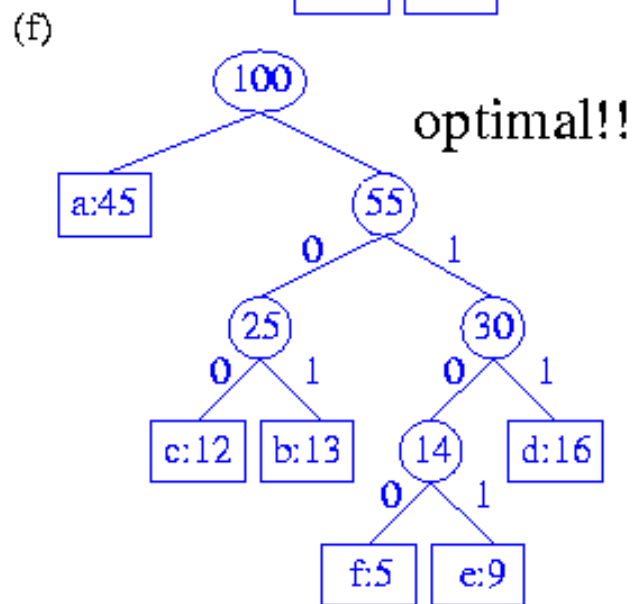
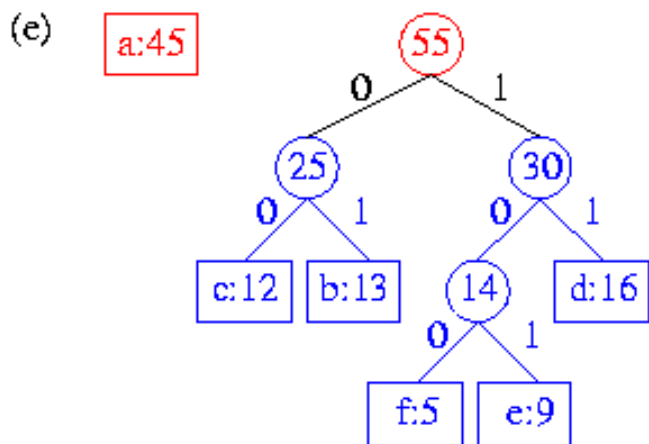
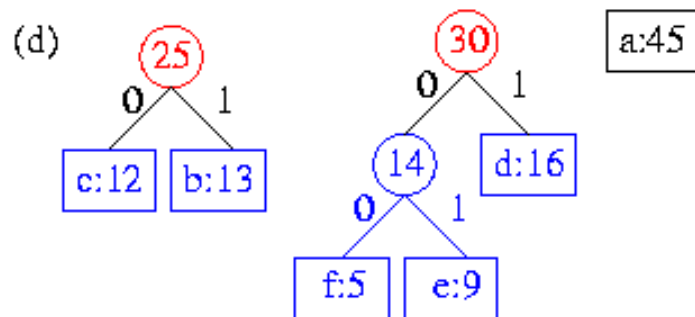
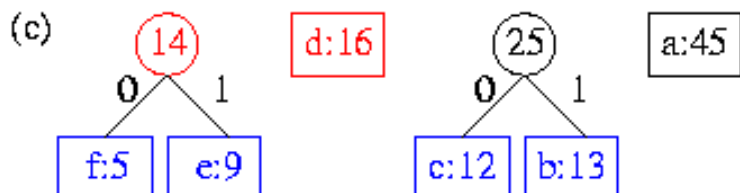
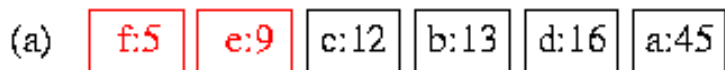
optimal code  $\rightarrow$  full binary tree!!



Variable-length cost = 224

# 哈夫曼编码过程

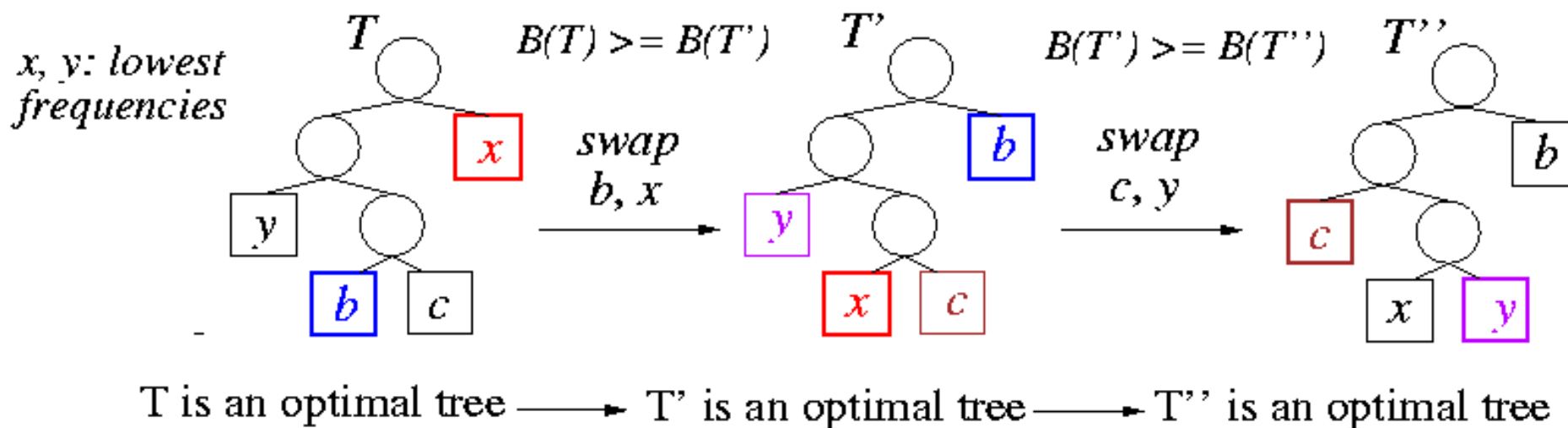
- 每步将费用最小的两个节点配对.



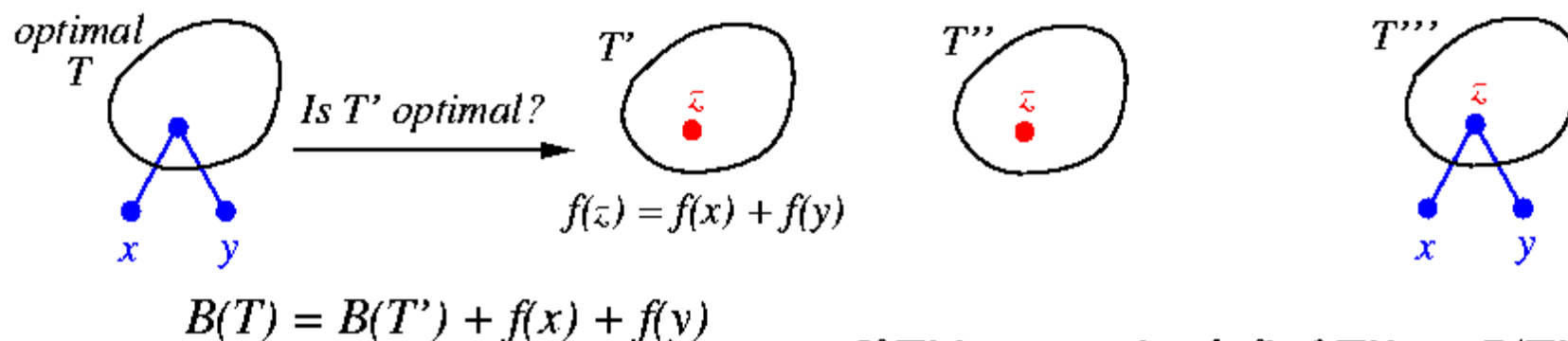
```
Huffman(C)
1.  $n \leftarrow |C|$ ;
2.  $Q \leftarrow C$ ;
3. for  $i \leftarrow 1$  to  $n-1$ 
4.    $z \leftarrow \text{Allocate-Node}()$ ;
5.    $x \leftarrow \text{left}[z] \leftarrow \text{Extract-Min}(Q)$ ;
6.    $y \leftarrow \text{right}[z] \leftarrow \text{Extract-Min}(Q)$ ;
7.    $f[z] \leftarrow f[x] + f[y]$ ;
8.    $\text{Insert}(Q, z)$ ;
9. return  $\text{Extract-Min}(Q)$ 
```

- 时间复杂度:  $O(n \lg n)$ .
  - $\text{Extract-Min}(Q)$ 堆操作要  $O(\lg n)$ 。
  - 开始需要  $O(n \lg n)$  的时间创建二项堆。

- 贪心选择:** 两个频率最低的字符  $x$  和  $y$  长度相同而且仅仅是最后一位不同。



- 最优子结构:** 令  $T$  为  $C$  的最优前缀码的二叉树。令  $z$  为两个叶子字符  $x$  和  $y$  的父亲。如果  $f[z] = f[x] + f[y]$ , 树  $T' = T - \{x, y\}$  表示  $C' = C - \{x, y\} \cup \{z\}$  的最优前缀码。



If  $T'$  is not optimal, find  $T''$  s.t.  $B(T'') < B(T')$ .

$z$  in  $C' \Rightarrow z$  is a leaf of  $T''$ .

Add  $x, y$  as  $z$ 's children ( $T'''$ )

$\Rightarrow B(T''') = B(T'') + f(x) + f(y)$

$< B(T') + f(x) + f(y)$

$= B(T)$  a contradiction!!

# 课堂练习：分数背包问题

- **背包问题**: 给定  $n$  物体, 第  $i$  个 价值  $v_i$  元 并且重量  $w_i$  磅, 一个贼希望带走价值尽量多的东西, 但是他的背包仅仅可以容纳  $W$  磅.
  - **0-1 背包问题**: 每个物体要么带着, 要么放下 (0-1 决定).
  - **分数 背包问题**: 允许拿走部分物体.
- 
- (1) 分数背包问题是否具有贪婪选择特征? 请给出证明。
  - (2) 若使用贪心算法求解分数背包问题, 请给出基本求解思路
  - (3) 请给出算法伪代码
  - (4) 0-1背包问题是否具有贪婪选择特征?

- 给定字符集 $C=\{a, b, c, d, e, f, g, h\}$ ，各字符出现的频率分别为： $\{83, 14, 28, 38, 131, 29, 20, 57\}$ ，求最优前缀码和平均码长。



合肥工业大学  
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队  
HFUT Data Mining and Intelligent Computing Laboratory